Apprentissage Statistique

Confrontation théorie et mise en œuvre Python

Objectifs

L'apprentissage statistique suppose souvent un monde idéal. Ce travail pratique vous confronte aux contraintes géométriques et algébriques réelles rencontrées par les algorithmes, caractérisées ici par trois difficultés majeures :

1. Une **topologie en anneau** (classes non linéairement séparables).
2. La présence de **valeurs aberrantes extrêmes asymétriques (*outliers*)**.
3. Le **fléau de la dimension** ($p > n$) lors de l'estimation de matrices de covariance.

Vous diagnostiquerez mathématiquement et visuellement l'échec de plusieurs algorithmes face à ces anomalies, puis vous implémenterez des stratégies (prétraitement, quantiles robustes, régularisation spectrale) pour restaurer la fiabilité du modèle.

Ne pas hésiter à critiquer l'énoncé.

1 Génération des données

Vous modélisez un système de diagnostic. Les pièces conformes (Classe 0) respectent les caractéristiques d'usine (situées au centre). Les pièces défectueuses (Classe 1) s'en écartent, formant une couronne. Parfois, les capteurs de mesure déraillent sur un seul des axes et produisent des valeurs géométriquement absurdes, labellisées à tort dans la Classe 0.

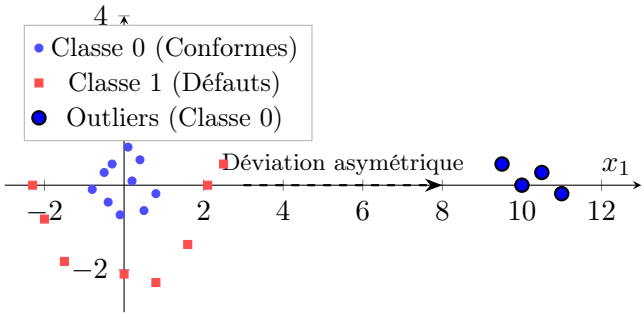

Figure 1 – Topologie du jeu de données d'apprentissage dans $\mathbb{R}^2$.

Travail à réaliser en Python

1. Utilisez la fonction `make_circles` de `sklearn.datasets` pour générer le jeu de données "sain" ($N = 400$, `noise=0.1`, `factor=0.3`).
   Indication : `make_circles` attribue par défaut la classe 1 au cercle intérieur. Pour coller à l'énoncé (Classe 0 au centre), inversez les labels via `y = 1 - y`.
2. Ajoutez manuellement 4 points aberrants (*outliers*) **asymétriques** pour la Classe 0 : les coordonnées seront situées autour de $x_1 = 25$ et $x_2 = 0$. Stockez le tout dans `X_dirty` et `y_dirty`.
3. Tracez le nuage de points sans forcer de limites sur les axes. Que constatez-vous visuellement concernant l'échelle globale et l'affichage des données saines ?

2 Modèles génératifs QDA et effondrement paramétrique

L'Analyse Discriminante Quadratique (QDA) modélise la densité de probabilité de chaque classe par une distribution Gaussienne $\mathcal{N}(\hat{\boldsymbol{\mu}}_k, \hat{\boldsymbol{\Sigma}}_k)$.

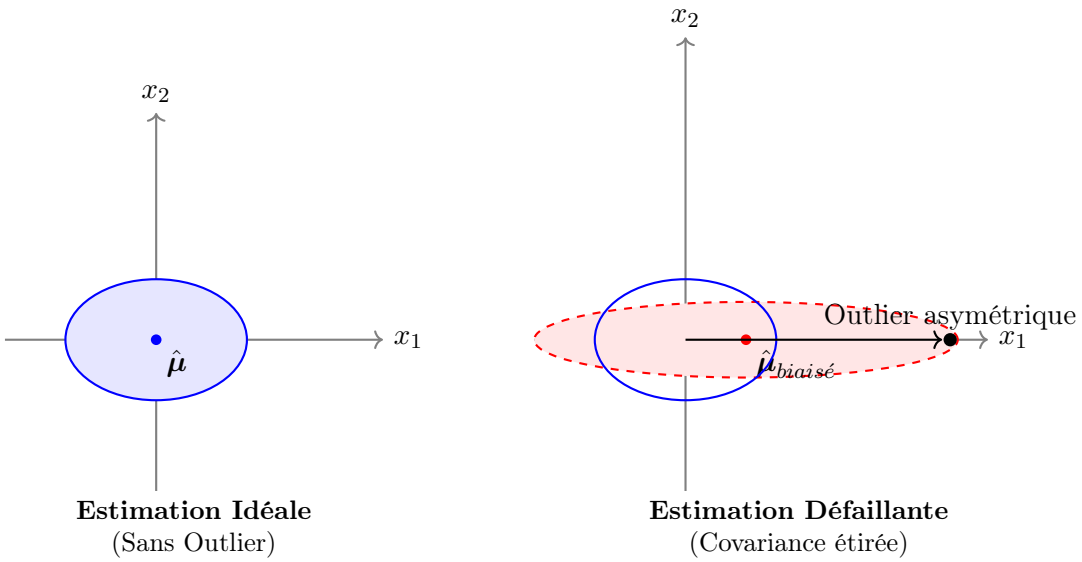

Figure 2 – Déformation de la moyenne $\hat{\boldsymbol{\mu}}$ et de la covariance $\hat{\boldsymbol{\Sigma}}$ causée par un outlier en QDA.

Travail à réaliser et Analyse

1. **Expérimentation :** entraînez un modèle `QuadraticDiscriminantAnalysis` sur les données `X_dirty`.
   Affichez la frontière de décision avec `DecisionBoundaryDisplay.from_estimator` en zoomant sur le centre (limites des axes à $[-3, 3]$).
   Le modèle a-t-il encerclé correctement les données saines ?
2. **Analyse théorique :** la matrice de covariance empirique est
   $$\hat{\boldsymbol{\Sigma}}_k = \frac{1}{N_k - 1} \sum_{i \in \mathcal{C}_k} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)^T.$$
   En vous appuyant sur cette équation et sur la distance de Mahalanobis, expliquez pourquoi la présence de seulement 4 outliers provoque la destruction totale de la frontière centrale.
3. **Analyse de LDA :** indépendamment du problème d'outlier ;
   pourquoi l'algorithme `LinearDiscriminantAnalysis` est-il théoriquement voué à l'échec sur ce dataset ?
4. **Prétraitement :** implémentez un modèle `IsolationForest` pour détecter et exclure les anomalies.
   Ré-entraînez QDA sur ces données nettoyées (`X_clean`).
   Tracez la nouvelle frontière (zoomée) et constatez la restauration.

3 k-NN, topologie locale et piège du prétraitement

Le k-Plus Proches Voisins (k-NN) n'estime pas de statistiques globales, il s'appuie uniquement sur le calcul des distances locales. Il est indispensable de normaliser les données avant d'utiliser le k-NN, mais le choix de l'outil est critique.

Travail à réaliser et analyse

1. **k-NN Brut :** entraînez un `KNeighborsClassifier` ($k = 5$) sur les données `X_dirty` non standardisées.

   Tracez la frontière (zoom sur $[-3, 3]$).

   Le k-NN subit-il la même déformation globale que le QDA ? Justifiez par le concept géométrique de "cellules de Voronoï".

2. **Le piège de la normalisation :**

   a) Utilisez un `Pipeline` scikit-learn combinant un `MinMaxScaler` et un `KNeighborsClassifier`. Entraînez-le sur `X_dirty`.

      Tracez la frontière en zoomant sur la région où les inliers se sont tassés (ex : `xlim(0, 0.15)` et `ylim(-0.1, 1.1)`).

   b) **Analyse théorique :** écrivez la formule du `MinMaxScaler`. Expliquez numériquement pourquoi cette méthode, face à des outliers asymétriques (extrêmes sur l'axe x_1 mais pas sur x_2), distord totalement la métrique euclidienne des données saines et transforme les cercles en bandes étirées.

3. **La solution robuste :** remplacez le scaler par un `RobustScaler` dans le `Pipeline`.

   Ré-entraînez et vérifiez la restauration parfaite de la frontière circulaire (zoom sur $[-3, 3]$).

   Pourquoi cet outil est-il immunisé ?

4 SVM, astuce du noyau et marge souple

Pour séparer nos cercles sans subir l'écrasement des modèles linéaires, nous utiliserons une **Machine à Vecteurs de Support (SVM)** dotée d'un noyau polynomial. Le problème d'optimisation primal du SVM à marge souple est défini par :

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N}\xi_i \quad \text{sous la contrainte : } y_i(\mathbf{w}^T\phi(\mathbf{x}_i)+b) \geq 1-\xi_i \quad (\text{avec } \xi_i \geq 0)$$

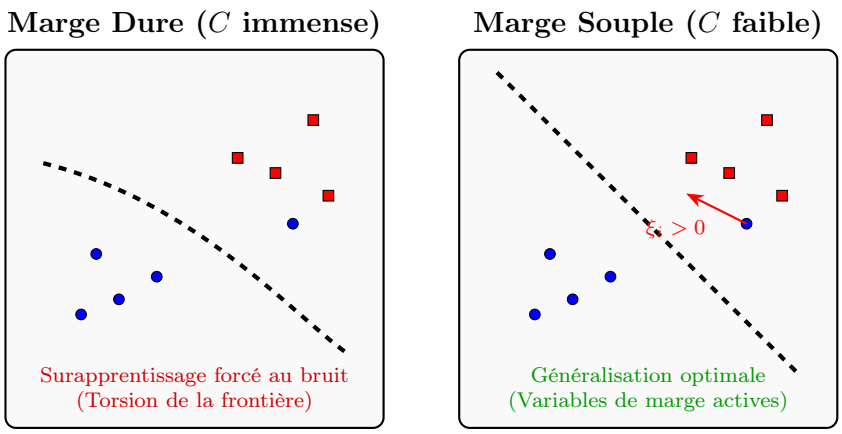

Figure 3 – Impact du paramètre de régularisation C sur la flexibilité de la frontière face à une donnée aberrante.

Travail à réaliser et analyse

1. **Astuce du Noyau (théorie)** : développez le noyau polynomial de degré 2 :

$$K(\mathbf{x},\mathbf{z}) = (\langle \mathbf{x},\mathbf{z}\rangle + 1)^2$$

   Déduisez-en conceptuellement pourquoi ce noyau permet à la SVM de séparer nos cercles en utilisant un simple hyperplan linéaire dans l'espace de redescription $\phi(\mathbf{x})$.

2. **Gestion des outliers via la marge** : créez deux `Pipeline`s combinant un `RobustScaler` et un `SVC`. Utilisez les données corrompues `X_dirty`.
   a) **Marge Dure** : `SVC(kernel='poly', degree=2, coef0=1.0, C=100000)`. (*Ajoutez `max_iter=10000` pour éviter un calcul infini de l'optimiseur*).
   Tracez la frontière (zoom sur $[-3, 3]$).
   b) **Marge Souple** : `SVC(kernel='poly', degree=2, coef0=1.0, C=0.05)`.
   Tracez la frontière.

3. **Analyse théorique** : en vous référant à l'équation primale ci-dessus et à la Figure 4, expliquez pourquoi la "Marge Dure" détruit la forme de la frontière pour tenter d'incorporer l'outlier.
   À l'inverse, quel est le rôle précis des variables d'écart (ξ_i) lorsque C est très faible ?

5 Fléau de la dimension et estimation de la covariance ($p > n$)

Dans les domaines de pointe en *Data Science* (ex : génomique), il est fréquent de posséder plus de variables explicatives que d'observations. Soit une matrice de *design* de données centrées $\mathbf{X}_c \in \mathbb{R}^{n \times p}$ où n est le nombre d'échantillons et p le nombre de variables, avec la condition critique $p > n$ (ex : $n = 50$ patients, $p = 200$ gènes).

La matrice de covariance empirique est calculée par :

$$\hat{\mathbf{\Sigma}} = \frac{1}{n-1} \mathbf{X}_c^T \mathbf{X}_c$$

Questions et travail à réaliser

1. **Simulation et rang (Python)** : générez une matrice $\mathbf{X}$ de $n = 50$ échantillons et $p = 200$ descripteurs issus d'une loi normale centrée.

   Calculez la matrice de covariance empirique correspondante $\hat{\mathbf{\Sigma}}$ (utilisez `rowvar=False` de `np.cov`).

   À l'aide de `numpy.linalg.matrix_rank`, calculez le rang empirique de cette matrice.

2. **Analyse algébrique** : le rang d'un produit matriciel est borné par la propriété : $\text{rang}(\mathbf{AB}) \leq \min(\text{rang}(\mathbf{A}), \text{rang}(\mathbf{B}))$.

   Démontrez formellement quel est le rang maximal atteignable par $\hat{\mathbf{\Sigma}}$.

   Déduisez-en pourquoi la matrice est impossible à inverser lorsque $p > n$.

   Quel sera l'impact direct sur un algorithme comme la QDA ?

3. **Spectre des valeurs propres (Python)** : calculez l'ensemble des valeurs propres de $\hat{\mathbf{\Sigma}}$ avec `numpy.linalg.eigh`.

   Combien de valeurs propres trouvez-vous numériquement nulles (aux erreurs d'arrondi flottant près, c'est-à-dire $< 10^{-10}$) ?

4. **Régularisation (Shrinkage de Ledoit-Wolf)** : pour contourner cette singularité algébrique, il est coutume de "contracter" la matrice vers l'identité via la formule :

   $$\hat{\mathbf{\Sigma}}_{LW} = (1-\alpha)\hat{\mathbf{\Sigma}} + \alpha \frac{\text{Tr}(\hat{\mathbf{\Sigma}})}{p} \mathbf{I}_p$$

   où $\alpha \in]0, 1]$ est la pénalité.

   Implémentez la classe `LedoitWolf` de `sklearn.covariance` sur $\mathbf{X}$.

   Vérifiez que le rang de la nouvelle matrice `lw.covariance_` est désormais bien égal à p.

5. **Mécanisme théorique** : expliquez algébriquement comment l'ajout du terme proportionnel à la matrice identité $\mathbf{I}_p$ modifie de façon salvatrice le spectre (les valeurs propres) de l'estimateur initial.